HS BO Hochschule Bochum	Hochschule Bochum Bochum University of Applied Sciences

Prüfungsform:	**Klausur**
Prüfungsfach:	**Objektorientierte Programmierung, Objektorientierte Programmiertechniken, Softwaretechnik und - Architektur 2 für Bachelor Wirtschaftsinformatik und Bachelor Informatik**
Semester:	**WiSe 2021 / 2022**
Datum:	**27.01.2022**
Prüfer:	**Prof. Dr. Ursula Oesing**
Bearbeitungsdauer:	**120 Minuten**
Punkte:	Es können 120 Punkte erreicht werden.

Hinweise:

Die 120 Punkte werden in Prozentanteile umgerechnet.
Mit der Abgabe der Klausur erklären Sie sich gesundheitlich in der Lage, die Klausur mitzuschreiben.
Mit der Abgabe der Klausur erklären Sie, dass Sie Ihre eingereichten Lösungen selbst und ohne Hilfe durch andere Personen erstellt haben.

Bei nachgewiesenen Täuschungsversuchen wird die Klausur mit 0 Prozent bzw. der Note 5.0 bewertet.
Die Missachtung der Themenzuordnung, siehe unten, führt zu massiven Punktabzügen, gegebenenfalls auch zu einer Nachprüfung in mündlicher Form.

Bitte beachten Sie die Hinweise zur Abgabe der Klausur des Dokuments Klausur_OP_SA2_HinweiseAbgabe_Themen.pdf.

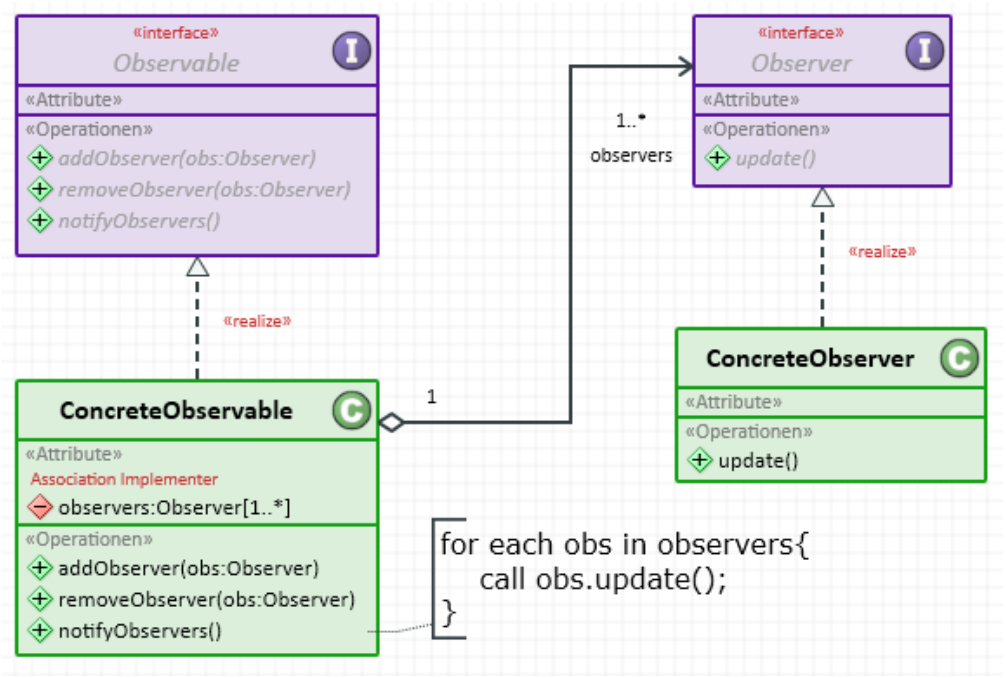
Bitte beachten Sie, dass die Aufgaben individualisiert sind. Jeder Teilnehmer und jede Teilnehmerin bearbeiten die Aufgaben für das eigene Thema. Bitte beachten Sie dazu die Zuteilung der Themen im Dokument Klausur_OP_SA2_HinweiseAbgabe_Themen.pdf.

1. Aufgabe: Versionierung / Versionsverwaltung (5 Punkte)

Gegeben ist zu Ihrem Thema zur Aufgabe 1 eine Frage. Beantworten Sie diese kurz (2-3 Sätze).

2. Aufgabe: Design Patterns (40 Punkte)

Gegeben sind ein Thema zur Aufgabe 2 und eines zu den Aufgaben 2 bis 5. Diese Aufgabe führen Sie zu den Themen zur Aufgabe 2 und zu den Aufgaben 2 bis 5 aus. Vorgestellt wird die Aufgabe mit dem Beispiel: *Personen.*
Gegeben ist weiterhin ein UML-Klassendiagramm zum Entwurfsmuster **Observer.**



Karl Eilebrecht, Gernot Starke, Patterns kompakt

Gegeben sind weiterhin die folgenden Klassen *PersonenModel, PersonenControl, PersonenView* und *Main* und eine Testausgabe in der Konsole.

```
import java.util.Vector;

public class PersonenModel {

   private Vector<String> personen = new Vector<>();

   public void lesePersonenAusDatei() {
         // anstatt Lesen aus Datei
         this.personen.add("Mustermann");
         this.personen.add("Musterfrau");
   }

   public String gibPersonenAus() {
         return this.personen.toString();
   }
}
```

```
public class PersonenControl {

   private PersonenModel personenModel;
   private PersonenView personenView;

   public PersonenControl() {
         this.personenModel = new PersonenModel();
         this.personenView = new PersonenView(this.personenModel, this);
         // Test
         this.personenModel.lesePersonenAusDatei();
         this.personenView.updateAnzeigePersonen();
   }
}

public class PersonenView {

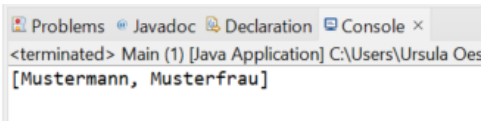
   PersonenModel personenModel;
   PersonenControl personenControl;

   public PersonenView(PersonenModel personenModel,
         PersonenControl personenControl) {
         this.personenModel = personenModel;
         this.personenControl = personenControl;
   }

   public void updateAnzeigePersonen() {
         System.out.println(
               this.personenModel.gibPersonenAus());
   }
}

public class Main {

   public static void main(String[] args) {
         new PersonenControl();
   }
}
```



Geben Sie den Quellcode an, der aus dem UML-Klassendiagramm zu dem Pattern *Observer* hervorgeht bzw. ergänzen Sie den vorliegenden Quellcode. **Passen Sie aber die Namen der Klassen Ihrem Thema an!** Übernehmen Sie also **nicht** die Namen der Klassen und Interfaces mit Ausnahme von *Observable*. Beispielsweise wird aus *Observer* wird *PersonenObserver.* Implementieren Sie die Methoden aus *ConcreteObservable.* und aus *ConcreteObserver*.
Geben Sie weiterhin den Quellcode an, der ergänzt werden muss, damit die Anwendung mit Hilfe von *Observer* die Konsolenausgabe erzeugt. Die Zeile *this.personenView.updateAnzeigePersonen();* aus PersonenControl entfällt dann. Geben Sie dabei an, wo der Quellcode ergänzt werden muss (Klasse, Methode).

3. Aufgabe: Vertiefung Java (30 Punkte)

Gegeben ist ein Thema zu den Aufgaben 2 bis 5. Diese Aufgabe führen Sie zu dem Thema zu den Aufgaben 2 bis 5 aus. Vorgestellt wird die Aufgabe mit dem Beispiel: *Person: Nachname, Vorname, Familienstand: ledig?.*

a) Generics, for each

Gehen Sie davon aus, dass eine Klasse *Person* existiert mit den Eigenschaften *nachname, vorname* und *familienstand.* Die Klasse enthält einen Konstruktor, welcher die Attribute mit Werten belegt, und get- und set-Methoden. Weiterhin enthält sie eine Methode ohne Parameter mit dem Rückgabetyp boolean, die *ledig?* überprüft. Geben Sie den Quellcode der zuletzt genannten Methode an. (Sie brauchen nicht den Quellcode der ganzen Klasse angeben.)

Geben Sie den Quellcode der main-Methode der Klasse *Anwendersystem* an, um ein Objekt vom Typ *ArrayList* unter Verwendung von Generics zu erzeugen. Die Liste soll zwei Objekte vom Typ *Person* beinhalten. Geben Sie den Quellcode unter Verwendung einer for each - Schleife an, so dass Sie die folgende Ausgabe erhalten. (Mit *for each* ist nicht die Verwendung einer Methode *forEach* aus gemeint!)

```
Musterfrau Elke: true
Mustermann Max: false
```

b) Funktionale Programmierung

Geben Sie den Quellcode an, um ein Objekt vom Typ *Person* zu erstellen und dann mit Hilfe dieses Objekts ein *BooleanSupplier* – Objekt unter Verwendung eines Lambda-Ausdrucks zu erstellen. Die Implementierung von *Boolean-Supplier* erfolgt mit Hilfe der Methode, welche *ledig?* überprüft, siehe oben.

Geben Sie dann den Quellcode an, um mit Hilfe des *BooleanSupplier* – Objekts die Ausgabe *true* oder *false* in der Konsole zu erhalten.

```
Ledig? true
```

Geben Sie weiterhin den Quellcode an, um mit Hilfe des *Person*-Objekts ein *Supplier<String>* - Objekt unter Verwendung eines Lambda-Ausdrucks zu erstellen. Die Implementierung von *Supplier<String>* erfolgt mit Hilfe einer bereits vorhandenen Methode der Klasse *Person.* Erzeugen Sie analog zu

```
Ledig? true
```

eine Konsolenausgabe.

Ist in dem *Supplier<String>* - Beispiel das funktionale Konzept *Pure Function* erfüllt? Begründen Sie Ihre Entscheidung.

4. Aufgabe: JUnit 5 und Zustandsbasierter Test (25 Punkte)

Gegeben ist ein Thema zu den Aufgaben 2 bis 5. Diese Aufgabe führen Sie zu dem Thema zu den Aufgaben 2 bis 5 aus. Vorgestellt wird die Aufgabe mit dem Beispiel: *Person: NACHNAME, Vorname, Familienstand: ledig?*

Gegeben sei die Klasse *Person* aus Aufgabe 3 a), eine Klasse mit den Eigenschaften *nachname, vorname* und *familienstand.* Die Klasse enthält einen Konstruktor, welcher die Attribute mit Werten belegt, und get- und set-Methoden. Weiterhin enthält sie die Methode mit dem Rückgabetyp boolean, die *ledig?* überprüft.

JUnit 5

Ergänzen Sie den vorliegenden Quellcode eines JUnit Tests. Die Methode der Klasse *Person,* die *ledig?* überprüft, soll durch diese JUnit Testklasse getestet werden. Implementieren Sie einen Testfall für das Ergebnis *true.* Benutzen Sie aus der Klasse Assertions
assertTrue(BooleanSupplier conditionSupplier, Supplier<String> messageSupplier).

```
import static org.junit.jupiter.api.Assertions.*;
import  org.junit.jupiter.api.*;
public class PersonTest{

   // Ergaenzen!!!



   @BeforeEach
   public void setUp() throws Exception{
         // Ergaenzen!!!

   }


   @AfterEach
   public void tearDown() throws Exception{
         // Ergaenzen!!!

   }

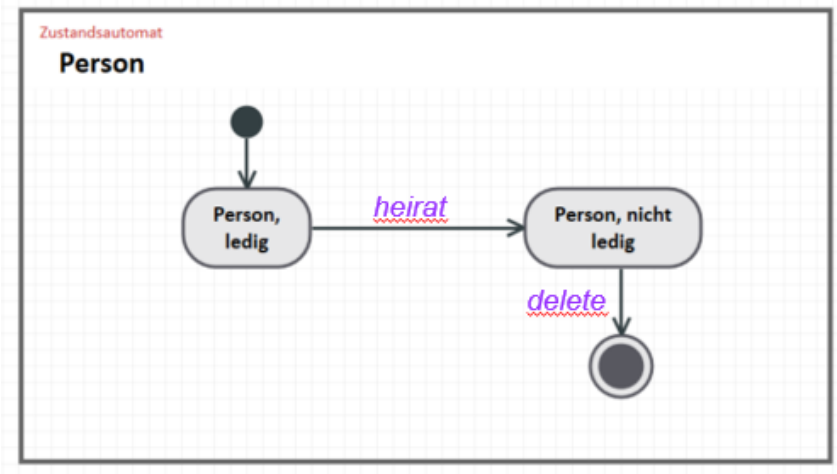
   @Test
   public void test() throws Exception{
         // Ergaenzen!!!


   }
}
```

Zustandsbasierter Test

Gegeben ist zusätzlich ein Thema zur Aufgabe 4, welches eine Transition vorgibt, hier im Beispiel *heirat*. Gegeben sei der folgende Zustandsautomat. (Hier werden weitere *Familienstände* außer *ledig* nicht differenziert.)

Geben Sie den analogen Zustandsautomaten für Ihre Thema an.
Geben Sie den Übergangsbaum an.
Geben Sie die Testfälle inklusive Sollergebnis an, die sich aus dem Übergangsbaum ergeben.



5. Aufgabe: SOLID und Clean Code (20 Punkte)

Gegeben sind ein Thema zu der Aufgabe 5 und ein Thema zu den Aufgaben 2 bis 5. Diese Aufgabe führen Sie zu den genannten Themen durch. Vorgestellt wird die Aufgabe mit dem Beispiel: Single Responsibility Principle und *Person: NACHNAME, Vorname, Familienstand: ledig?.*

a) Erläutern Sie das genannte SOLID Prinzip. Nennen Sie ein Beispiel unter Verwendung des Themas (2-3 Sätze). Beispiel:
Jede Klasse sollte eine strikt abgegrenzte Verantwortlichkeit besitzen. Die Klasse *Person* sollte also nur Personendaten, keine weiteren Daten wie beispielsweise Daten zu einer Adresse verwalten.

b) Wieso trägt das genannte SOLID Prinzip zu einer höheren Wartbarkeit und Lebensdauer von Software und / oder zu einer Reduktion der Fehleranfälligkeit bei (2-3 Sätze)?

c) Erläutern Sie kurz den genannten Wert aus Clean Code. Nennen Sie ein Beispiel unter Verwendung des Themas (2-3 Sätze) analog zum Beispiel in a).

d) Erläutern Sie kurz, wie SOLID und Clean Code zusammenhängen (2-3 Sätze). Erwähnen Sie die Grade.